Smart Pothole Detection and Road Monitoring System

FARAH MANIKINDI

SHARADHI CP

BANI V

HARSHITHA GR

Abstract

This project presents an ESP32-based prototype for detecting road depressions and displaying road-condition information on a web dashboard. The system combines an ultrasonic sensor, a magnetic sensor, a moisture sensor, and a GPS module. The ultrasonic sensor estimates the depth of a depression relative to a configured reference distance. The magnetic sensor detects wheel rotations, which are used to estimate travel distance and speed. The moisture sensor provides a threshold-based wet or dry indication.

The ESP32 processes the sensor readings and sends data over Wi-Fi to a Flask server using HTTP requests. The server maintains the latest readings, creates pothole tickets, and provides information to a browser-based dashboard. The dashboard displays sensor values, pothole counts, severity categories, moisture status, and maintenance progress.

The firmware uses predefined coordinates when a recent GPS fix is unavailable. These coordinates support indoor demonstrations and must be distinguished from actual measurements. The current implementation stores records in memory and is intended as a laboratory prototype. Experimental accuracy and field performance must be established through calibration and testing.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope 2

2 System Design **3**

2.1 System Overview 3

2.2 Operating Sequence 4

2.3 Hardware Components 5

2.4 Software Components 5

3 Hardware Implementation **6**

3.1 Pin Configuration 6

3.2 Ultrasonic Measurement 6

3.3 Depth Estimation 7

3.4 Wheel Rotation Measurement 7

3.5 Speed and Distance Calculation 8

3.6 Moisture Detection 8

3.7 GPS Operation 9

4 Software Implementation **10**

4.1 Embedded Firmware 10
4.2 Data Transmission 10
4.3 Flask Backend 11
4.4 API Endpoints 12
4.5 Severity Classification 12
4.6 Web Dashboard 12
4.7 Prototype Road-Health Indicator 13

5 Installation and Operation **15**
5.1 Project Files 15
5.2 Starting the Backend 15
5.3 Configuring the ESP32 16

6 Limitations and Future Improvements **17**
6.1 Current Limitations 17
6.2 Future Improvements 17

7 Conclusion **19**

A Project Links **20**

B Source Code Files **21**

List of Figures

2.1 System architecture showing sensors, ESP32, Flask server, and web dashboard. 4

3.1 Assembled sensing prototype. 9

4.1 Dashboard showing sensor readings and summary information. 13

4.2 Maintenance records and repair-status controls. 14

List of Tables

2.1 Main hardware components 5

3.1 ESP32 signal-pin assignments 6

4.1 Configured firmware timing intervals 10

4.2 Backend API endpoints 12

4.3 Severity rules in the current backend 12

Chapter 1

Introduction

1.1 Background

Road-condition monitoring helps identify damaged sections that require inspection and maintenance. Manual inspection requires personnel to observe road surfaces and record the location and condition of defects. A sensing system can assist this process by collecting measurements and presenting them in a structured form.

This project explores a small-scale sensing approach using an ESP32 microcontroller. Measurements from several sensors are combined with location information and displayed through a web interface.

1.2 Problem Statement

The project addresses the need for a prototype that can detect surface depressions, associate them with location information, and maintain a simple record of repair status.

The main challenge is to connect physical measurements with a software interface that makes detected events easy to view and manage.

1.3 Objectives

- Estimate depression depth using an ultrasonic sensor.
- Detect wheel rotations using a magnetic sensor.
- Calculate approximate wheel-based speed and distance.
- Identify wet or dry conditions using a moisture sensor.
- Obtain GPS coordinates when a recent fix is available.

- Send sensor readings from the ESP32 to a Flask server.
- Display live readings and pothole records on a website.
- Provide a basic maintenance-status update facility.

1.4 Scope

The implementation is a local-network prototype. It includes embedded sensing, Wi-Fi communication, a Python backend, and a browser interface.

It does not currently include permanent database storage, user authentication, production hosting, or validated road-survey accuracy. Simulated GPS positions are available for indoor demonstrations.

Chapter 2

System Design

2.1 System Overview

The system contains three main parts:

1. **Sensing unit:** Sensors connected to the ESP32 collect measurements.
2. **Processing and communication unit:** The ESP32 processes readings and sends JSON data over Wi-Fi.
3. **Monitoring application:** A Flask server receives the data and supplies it to a web dashboard.

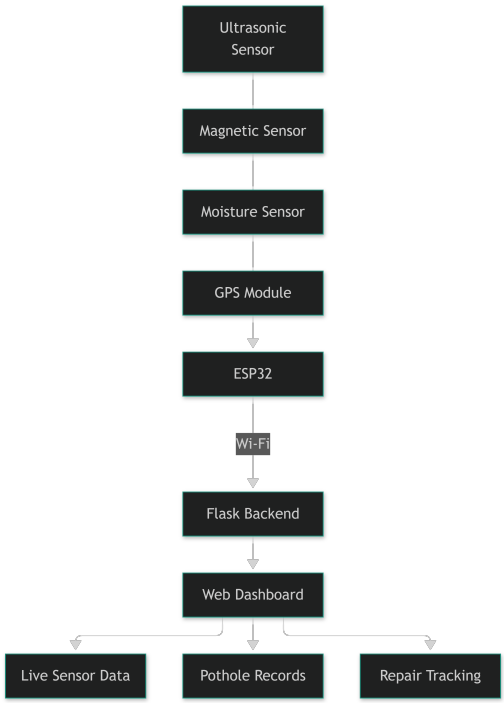


Figure 2.1: System architecture showing sensors, ESP32, Flask server, and web dashboard.

2.2 Operating Sequence

1. The ESP32 initializes its inputs, outputs, serial interfaces, and Wi-Fi connection.
2. GPS messages are processed to obtain a recent location fix when available.
3. The magnetic sensor is sampled to detect wheel rotations.
4. Ultrasonic and moisture measurements are updated periodically.
5. Depression depth and pothole detection status are calculated.
6. Sensor information is sent to the Flask server.
7. The backend updates live readings and creates a ticket when it observes a new detection event.
8. The browser retrieves updated information and refreshes the dashboard.

2.3 Hardware Components

Table 2.1: Main hardware components

Component	Purpose
ESP32	Sensor processing and Wi-Fi communication
Ultrasonic sensor	Distance measurement below the sensor
Magnetic sensor	Detection of wheel rotation events
Moisture sensor	Threshold-based wet or dry indication
GPS module	Position information when a fix is available
Wheel and magnet	Mechanical arrangement for rotation sensing
Power supply	Supply appropriate power to the hardware
Computer	Run the Flask backend and browser dashboard

2.4 Software Components

- Arduino IDE for compiling and uploading firmware.
- ESP32 Arduino board support.
- TinyGPSPlus for parsing GPS data.
- WiFi and HTTPClient libraries for communication.
- Python and Flask for the backend.
- HTML, CSS, and JavaScript for the dashboard.
- Git and GitHub for version control and project sharing.

Chapter 3

Hardware Implementation

3.1 Pin Configuration

Table 3.1 shows the GPIO assignments defined in the uploaded firmware.

Table 3.1: ESP32 signal-pin assignments

Signal	**ESP32 Pin**
Ultrasonic trigger	GPIO 12
Ultrasonic echo	GPIO 13
Magnetic sensor analog output	GPIO 35
Moisture sensor analog output	GPIO 34
GPS transmit signal to ESP32 receive input	GPIO 16

The GPS serial interface is configured at 9600 baud. The firmware does not configure an ESP32 transmit pin for this GPS connection.

These assignments describe signal connections only. Power requirements and signal voltage compatibility must be checked against the actual sensor modules used.

3.2 Ultrasonic Measurement

The ESP32 sends a trigger pulse and measures the duration of the returned echo pulse. The firmware calculates distance as

$$d = \frac{t \times 0.0343}{2} \tag{3.1}$$

where d is the estimated distance in centimetres and t is the echo duration in microsec-

onds.

The division by two accounts for the outward and return travel of the sound pulse.

The code rejects measurements below 2 cm or above 400 cm. It also treats an echo timeout as an invalid reading.

3.3 Depth Estimation

The estimated depression depth is calculated as

$$h = \max(0, d - d_0) \tag{3.2}$$

where h is the estimated depth, d is the measured distance, and d_0 is the configured normal-road reference distance.

The current firmware uses

$$d_0 = 2.0 \text{ cm} \tag{3.3}$$

and identifies a pothole condition when

$$h > 0.5 \text{ cm}. \tag{3.4}$$

These values are prototype settings. The reference distance must match the actual mounting arrangement, and the detection threshold should be evaluated experimentally.

3.4 Wheel Rotation Measurement

A magnet mounted on the wheel changes the magnetic sensor output as it passes the sensor.

The firmware uses separate detection and release thresholds, consecutive-sample checks, and a minimum interval between counted events. These measures aim to reduce repeated counts from a single magnet passage.

The speed calculation assumes that one counted event corresponds to one wheel revolution. This assumption must match the mechanical arrangement.

3.5 Speed and Distance Calculation

The configured wheel diameter is

$$D = 3.2 \text{ cm}. \tag{3.5}$$

The wheel circumference is

$$C = \pi D. \tag{3.6}$$

For ΔN new rotations during an interval Δt in seconds, the estimated speed is

$$v = \frac{\Delta N C}{\Delta t}. \tag{3.7}$$

With the configured dimensions, the calculated speed is in centimetres per second.

The estimated travel distance is

$$s = NC \tag{3.8}$$

where N is the total counted rotations. The firmware calculates this distance locally; the current JSON payload does not include a distance field.

3.6 Moisture Detection

The firmware reads the moisture sensor through an analog input. Its current classification is

$$\text{Status} = \begin{cases} \text{WET}, & M > 1800 \\ \text{DRY}, & M \leq 1800 \end{cases} \tag{3.9}$$

where M is the raw analog reading.

The threshold and signal direction must be checked using the actual sensor. The wet or dry indication is not a measurement of water depth.

3.7 GPS Operation

A GPS position is used when the location is valid and its reported age is less than five seconds.

If this condition is not met, the firmware uses one of four predefined coordinates. The transmitted GPS status identifies the source as `LIVE` or `SIMULATED`.

Simulated coordinates support demonstrations but do not establish the physical location of a detected depression.

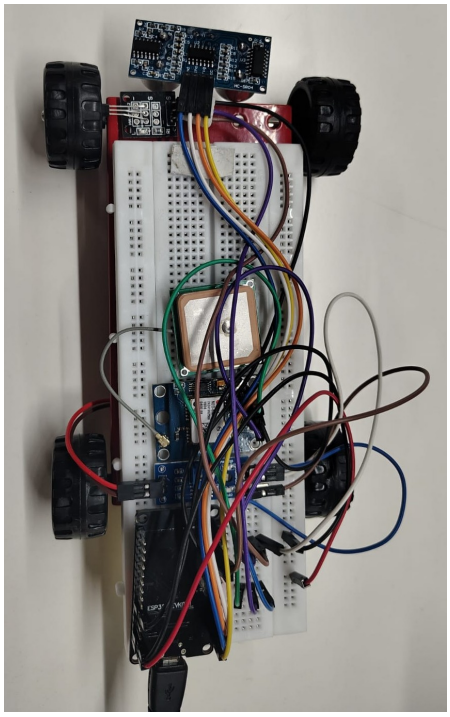


Figure 3.1: Assembled sensing prototype.

Chapter 4

Software Implementation

4.1 Embedded Firmware

The ESP32 firmware performs sensor acquisition, event detection, calculations, and communication.

The configured timing intervals are shown in Table 4.1.

Table 4.1: Configured firmware timing intervals

Operation	**Interval**
Ultrasonic and moisture updates	500 ms
Speed calculation	1000 ms
Sensor-data transmission	2000 ms
Serial summary display	5000 ms
Wi-Fi retry interval	10000 ms

These are configured scheduling intervals. Blocking operations, including network requests and echo measurement, can affect the actual time between operations.

4.2 Data Transmission

The ESP32 sends an HTTP POST request to

`http://LAPTOP_IP:5001/api/sensor`

The request body contains JSON data. An illustrative payload is shown below; it is not an experimental result.

```
{
  "ultrasonic": 6.5,
  "depth": 4.5,
  "magnetic": 2100,
  "rotations": 12,
  "speed": 10.05,
  "moisture": "DRY",
  "gps_status": "SIMULATED",
  "latitude": 12.971600,
  "longitude": 77.594600,
  "pothole": true
}
```


4.3 Flask Backend

The backend maintains the latest sensor readings and a list of pothole records. A thread lock protects shared data during updates and reads.

A new ticket is created when the received pothole status changes from false to true. Consecutive true values do not create additional tickets until the server receives a false value followed by another true value.

Each ticket contains:

- Ticket identifier.
- Latitude and longitude.
- GPS source status.
- Estimated depth.
- Severity category.
- Wet or dry status.
- Maintenance status.
- Detection timestamp.

Records are stored in memory and are cleared when the server process restarts.

4.4 API Endpoints

Table 4.2: Backend API endpoints

Method	Endpoint	Purpose
POST	`/api/sensor`	Receive sensor data
GET	`/api/sensor`	Read latest sensor data
GET	`/api/potholes`	Read pothole tickets
GET	`/api/dashboard`	Read summary statistics
PUT	`/api/maintenance/<id>`	Update repair status
POST	`/api/reset`	Clear pothole records

4.5 Severity Classification

The backend applies the following rules in descending order.

Table 4.3: Severity rules in the current backend

Depth	Returned Category
$h > 5$ cm	Severe
$3 < h \leq 5$ cm	Moderate
$1 < h \leq 3$ cm	Minor
$h \leq 1$ cm	No category returned

The firmware can detect depths greater than 0.5 cm, but the backend does not currently return a category for depths at or below 1 cm. This mismatch should be resolved before using the classifications for evaluation.

The categories are project-defined thresholds and are not presented as an official road-maintenance standard.

4.6 Web Dashboard

The browser interface displays:

- Latest sensor measurements.
- Total recorded potholes.
- Severe pothole count.
- Wet-condition pothole count.
- Pending and completed repairs.

- A prototype road-health indicator.
- Maintenance records and completion controls.
- Summary counts by severity and moisture status.

The JavaScript requests live sensor data approximately every two seconds. Dashboard, maintenance, and analytics information is refreshed approximately every three seconds.

4.7 Prototype Road-Health Indicator

When records are available, the backend calculates

$$R = \max(0, 100 - 10\bar{h}) \tag{4.1}$$

where $\bar{h}$ is the average recorded depth in centimetres.

When there are no records, the displayed value is 100. This is a software default, not evidence that a road has been inspected or found free of defects.

The indicator is a demonstration metric and is not a validated pavement-condition index.

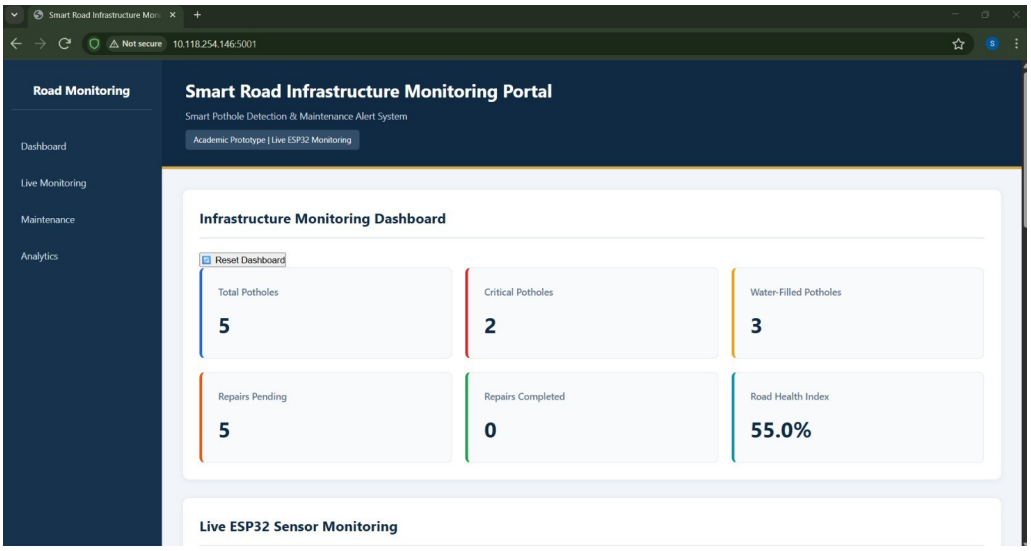


Figure 4.1: Dashboard showing sensor readings and summary information.

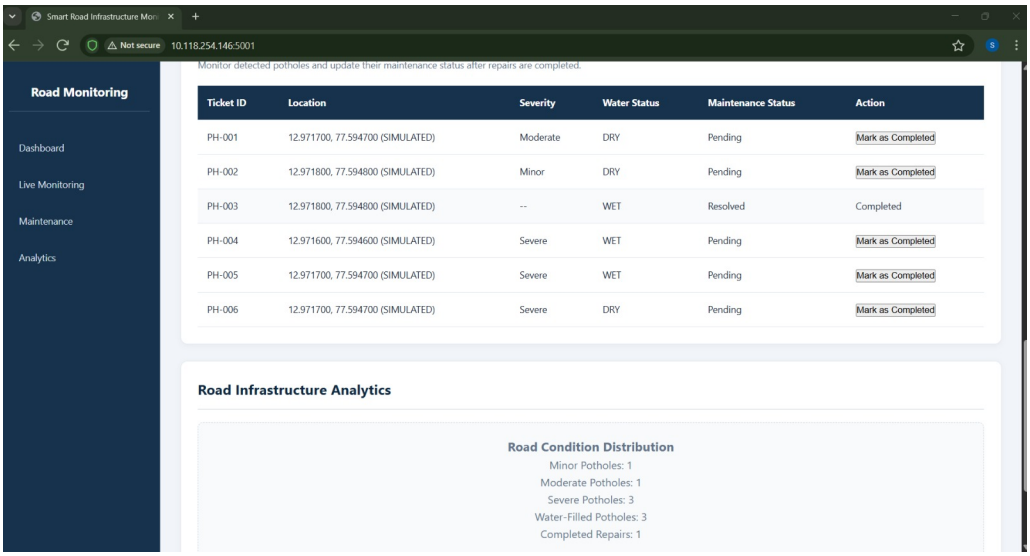


Figure 4.2: Maintenance records and repair-status controls.

Chapter 5

Installation and Operation

5.1 Project Files

The website files should remain in the same directory because the Flask backend serves them from that location.

```
app.py
index.html
style.css
script.js
requirements.txt
README.md
.gitignore
esp32_final_code/esp32_final_code.ino
static/images/
static/videos/
```

5.2 Starting the Backend

Create a `requirements.txt` file containing:

```
Flask
```

From Windows Command Prompt in the project directory, run:

```
py -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python app.py
```

Open the following address in a browser:

```
http://localhost:5001
```

The Python process must remain running while the dashboard is in use.

5.3 Configuring the ESP32

1. Install ESP32 board support in Arduino IDE.
2. Install the TinyGPSPlus library.
3. Select the appropriate board and serial port.
4. Enter the local Wi-Fi name and password.
5. Set the server URL to the laptop's local IPv4 address using port 5001.
6. Compile and upload the sketch.
7. Open Serial Monitor at 115200 baud.

The laptop and ESP32 must have network connectivity to each other. A successful sensor request should produce HTTP response code 200 in the Serial Monitor.

Chapter 6

Limitations and Future Improvements

6.1 Current Limitations

- Sensor mounting, tilt, and surface geometry can influence the ultrasonic measurement.
- The reference distance and sensor thresholds require calibration for the actual hardware.
- Simulated GPS coordinates do not represent measured detection locations.
- Records are not preserved after a server restart.
- The backend has no user authentication.
- The severity function does not classify detected depths between 0.5 cm and 1 cm.
- A detection that occurs and clears between data transmissions may not reach the backend.
- Separate detections can be merged into one ticket if the backend does not receive a false state between them.
- Network requests can delay other firmware operations.
- Wheel slip or missed magnetic events can affect speed and distance estimates.
- The road-health value is a demonstration formula.

6.2 Future Improvements

- Add persistent storage using a database.
- Store individual detection events in a queue until the server acknowledges them.
- Assign a unique identifier and timestamp to each event.
- Align detection and severity thresholds.

- Validate incoming sensor values at the backend.
- Add location-based duplicate handling.
- Plot verified GPS locations on a map.
- Add authentication and suitable deployment controls.
- Improve sensor mounting and calibration.
- Compare repeated measurements with known depths.
- Evaluate operation at different travel speeds.

Chapter 7

Conclusion

The project implements an ESP32-based sensing prototype connected to a Flask web application. It combines ultrasonic distance measurement, magnetic rotation sensing, moisture classification, and GPS information to support pothole monitoring.

The software provides live sensor access, event records, severity categories, and maintenance-status updates. The implementation demonstrates the integration of embedded hardware with a web dashboard.

Its practical accuracy and reliability remain dependent on calibration and experimental verification. Persistent storage, reliable event delivery, consistent classification, and validated location data are the main improvements needed for a more complete system.

Appendix A

Project Links

- GitHub repository: *click here for link*

Appendix B

Source Code Files

The implementation discussed in this report consists of:

- `esp32_final_code.ino`: Embedded sensing and communication.
- `app.py`: Flask routes, records, and maintenance management.
- `index.html`: Dashboard structure.
- `style.css`: Dashboard styling.
- `script.js`: API requests and interface updates.

If a complete code appendix is required, upload the source files to Overleaf and uncomment the relevant commands below.